

Task 1: __str__ and __repr__ (Book Class)

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def __str__(self):
        return f"Book: {self.title} written by {self.author}"

    def __repr__(self):
        return f"Book(title='{self.title}', author='{self.author}', price={self.price})"

b1 = Book("Python Basics", "Guido", 500)
b2 = Book("AI World", "Sam", 800)

print(b1)
print([b1, b2])
```

Task 2: __eq__ (Mobile Class)

```
class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def __eq__(self, other):
        return self.brand == other.brand and self.model == other.model

m1 = Mobile("Apple", "iPhone 15", 80000)
m2 = Mobile("Apple", "iPhone 15", 85000)
m3 = Mobile("Samsung", "S24", 90000)

print(m1 == m2)
print(m1 == m3)
```

Task 3: __new__ and __init__ (User Class)

```
class User:
    def __new__(cls, name):
        print("1. Object is being created (Memory Allocated)")
        return super().__new__(cls)

    def __init__(self, name):
        print("2. Object is initialized (Variables Set)")
        self.name = name

u1 = User("Nagendra")
```

Task 4: __enter__ and __exit__ (Context Manager)

```
class DatabaseConnection:
    def __enter__(self):
        print("Database Connected")
```

```
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Database Closed")

with DatabaseConnection():
    print("Performing Query...")
```

Task 5: __call__ (Calculator Class)

```
class Calculator:
    def __call__(self, a, b):
        return a + b

calc = Calculator()
result = calc(10, 20)
print("Sum is:", result)
```

Task 6: __getitem__ and __setitem__ (ShoppingCart)

```
class ShoppingCart:
    def __init__(self):
        self.items = ["Apple", "Banana", "Milk"]

    def __getitem__(self, index):
        return self.items[index]

    def __setitem__(self, index, value):
        self.items[index] = value

cart = ShoppingCart()
print(cart[1])
cart[1] = "Mango"
print(cart[1])
```

Task 7: __del__ (Destructor)

```
class Session:
    def __init__(self, user):
        self.user = user
        print(f"Session started for {self.user}")

    def __del__(self):
        print("Session Ended")

s1 = Session("Admin")
del s1
```

Task 8: __contains__ (Membership)

```
class Library:
    def __init__(self):
        self.books = ["Python", "Java", "C++"]

    def __contains__(self, item):
```

```
return item in self.books
```

```
lib = Library()
print("Python" in lib)
print("Ruby" in lib)
```

Task 9: __gt__ and __lt__ (Comparison)

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __gt__(self, other):
        return self.salary > other.salary

    def __lt__(self, other):
        return self.salary < other.salary

e1 = Employee("Alice", 50000)
e2 = Employee("Bob", 30000)

if e1 > e2:
    print(f"{e1.name} earns more than {e2.name}")
```

Task 10: __iter__ and __next__ (Iterator)

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= self.end:
            num = self.current
            self.current += 1
            return num
        else:
            raise StopIteration

for i in Counter(1, 5):
    print(i)
```